

CHAPITRE 2

Manipuler l'outil de mesure de la qualité du code (SonarQube)

1. **Notions des métriques de la qualité du code**
2. Présentation de SonarQube
3. Installation de SonarQube
4. Analyse de code source avec SonarQube
5. Intégration de SonarQube avec les outils ALM et Configuration



02-Manipuler l'outil de mesure de la qualité du code(SonarQube) :

Notions de métriques

Définitions

- **Une métrique** : est une caractéristique (ou une propriété) d'une application.
- **Métrique logicielle** : mesure d'une propriété d'un logiciel (par exemple le nombre de lignes de codes).
- **Exemples de métriques** :
 - Lignes de codes
 - Nombre de méthodes par classe
 - Couplage afférent/efférent
 - Niveau d'abstraction
 - Indice Instabilité
 - Complexité cyclomatique(compter le nombre de points de décision (if, case, while, ...)).
- **Pas de métrique "absolue"** : la pertinence de chaque métrique dépend du projet et surtout de l'interprétation qui en est faite.

02-Manipuler l'outil de mesure de la qualité du code(SonarQube) :

Notions de métriques

Définitions

- On distingue deux types de qualités :
 - La **qualité du code (qualité structurelle du logiciel)** : concerne la manière avec laquelle une fonctionnalité est implémentée, et inclut entre autres la robustesse, la maintenabilité, la lisibilité ou encore l'évolutivité.
 - La **qualité d'un logiciel (qualité fonctionnelle)**: concerne le résultat final.
- la **mesure de la qualité du code** peut en partie être mesurée automatiquement par des outils d'analyse statique du code source. Ces outils sont disponibles pour la plupart des langages et proposent de **nombreuses métriques** et conventions standards.
- La **qualité du code** peut être également évaluée par d'autres critères comme :
 - L'architecture qui inclut la maintenabilité, l'évolutivité, la performance et la pertinence
 - La documentation,
 - La portabilité et la sécurité.
 - La fiabilité
 - Le nombre de bugs connus.
- Tous ces critères sont fortement dépendants du contexte comme la spécificité du logiciel ou programme au niveau fonctionnel, son avenir mais aussi d'autres facteurs importants comme le marché et ses utilisateurs.

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Exemples d'outils



Exemples d'outil pour analyser le code:

Cobertura

Crap4J

PMD

FindBugs

Eclipse Metrics

Jdepend

Review Board

Crucible

Raxis

Technologies RIPS

PVS-Studio

Kiuwan

CodeScene

Expert visuel

Veracode

CodeSonar

SonarQube

CHAPITRE 2

Manipuler l'outil de mesure de la qualité du code (SonarQube)

1. Notions des métriques de la qualité du code
- 2. Présentation de SonarQube**
3. Installation de SonarQube
4. Intégration de SonarQube avec les outils ALM et Configuration
5. Analyse de code source avec SonarQube



02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Présentation de SonarQube

C'est quoi l'outil SonarQube ?

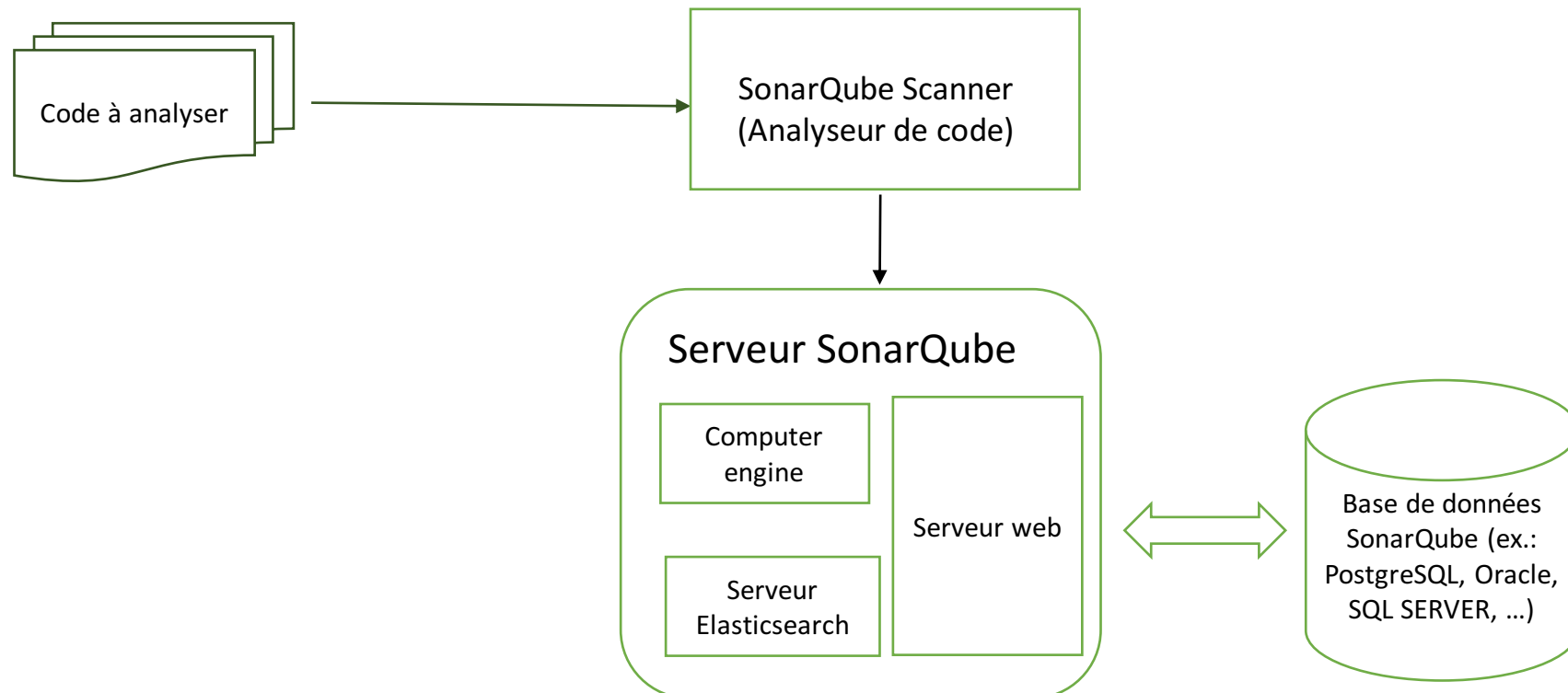
- **SonarQube** est un logiciel open source de mesure de la qualité du code source de projets de développement.
- Il permet d'obtenir des informations sur la qualité au niveau du projet, du fichier ou d'un module et donne des indications sur chaque problème de qualité détecté et le temps de remédiation.
- Son périmètre est le code source, le design ainsi que les tests unitaires, il supporte plus d'une vingtaine de langages de programmation, dont **C/C++,C#,Java, Python, PHP, JavaScript..**

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Présentation de SonarQube

Architecture de SonarQube ?

- D'un point de vue architectural, Sonar est composé de plusieurs couches :
 - Un exécuteur dont le but sera de lancer un certain nombre d'outils d'analyse, et d'en agréger les résultats ;
 - Le serveur web qui permet la navigation et la consultation des analyses réalisées sur les projets ;
 - Une base de données, qui stocke et historise les informations sur les projets surveillés par Sonar ;



02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Présentation de SonarQube

Les types de métriques avec SonarQube?

- **SonarQube** génère un rapport consultable via un navigateur sur :
 - ✓ Densité des commentaires
 - ✓ Taux de couverture des tests unitaires
 - ✓ des conventions de nommage
 - ✓ Respect des règles de codage et des bonnes pratiques
 - ✓ Détection de bogues
 - ✓ Détection de code mort
 - ✓ Détection de code dupliqué
 - ✓ Complexité, complexité cognitive)
 - ✓ Score de maintenabilité, fiabilité et sécurité du code (complexité cyclomatique)
 - ✓ Dette technique (estimation du temps nécessaire pour fixer tous les problèmes détectés)
 - ✓ Sonar couvre les 7 axes de la qualité du code : **architecture & design , documentation ,respect des standards de codage ,non duplication du code , tests unitaires ,complexité ,bogues potentiels.**
- Il est possible d'automatiser l'analyse avec un serveur d'intégration continue (**ex. Jenkins, Travis**).

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) : Présentation de SonarQube

Tableau du bord avec SonarQube?

- **SonarQube** classe les défauts logiciels selon **3** catégories :
 - Les **bugs** : anomalies évidentes du code. Ils impactent la **fiabilité** (reliability) de l'application.
 - Les **vulnérabilités** : faiblesses du code pouvant nuire au système. Elles impactent la **sécurité** de l'application.
 - Les **code smells** : anti-patterns (ou anti-patterns). Ils impactent la **maintenabilité** de l'application ,en général les défauts pratiques dans le code d'un programme comme code dupliqué , répété , commentaires non nécessaires ...
- **SonarQube** affiche un tableau de bord résumé à l'exécution d'un projet montrant les scores de maintenabilité, fiabilité et sécurité, le taux de couverture des tests, le taux de duplications, la taille du projet ainsi que les langages des sources.



Exemple tableau du bord

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Exemples de métriques avec Sonarqube



1. Scores de maintenabilité, fiabilité et sécurité (MFS):

Score	Fiabilité	Sécurité	Maintenabilité
A	0 bug	0 vulnérabilité	$0,00 \leq \text{ratio dette technique} \leq 0,05$
B	Au moins 1 bug mineur	Au moins 1 vulnérabilité mineure	$0,06 \leq \text{ratio dette technique} \leq 0,1$
C	Au moins 1 bug majeur	Au moins 1 vulnérabilité majeure	$0,11 \leq \text{ratio dette technique} \leq 0,20$
D	Au moins 1 bug critique	Au moins 1 vulnérabilité critique	$0,21 \leq \text{ratio dette technique} \leq 0,50$
E	Au moins 1 bug bloquant	Au moins 1 vulnérabilité bloquante	$0,51 \leq \text{ratio dette technique} \leq 1$

Tableau montrant comment calculer les scores de MFS

(*) Le **ratio de la dette technique** est le ratio entre le coût pour remédier aux problèmes de type code smell et le coût de développement de l'application.

La formule de calcul est la suivante : **coût total de remédiation des issues / (coût pour développer une ligne de code x nombre de lignes de code)**
L'idée est de déterminer si réécrire l'application est plus rentable que corriger tous les problèmes. Lorsque le ratio est trop grand, il est préférable de réécrire l'application de zéro plutôt que d'essayer de réduire la dette en corrigeant les problèmes.
Le nombre de minutes pour remédier à chaque issue est stocké dans la base de données.

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Exemples de métriques avec Sonarqube

2. Mesures pour la documentation

- Quelques variables utilisés :
 - **Comment lines** = nombre de lignes contenant des commentaires de documentation ou du code commenté, les lignes de commentaires non significatives sont ignorées (ligne vide ou avec le caractère spécial *)
 - **Comments (%)** = densité de commentaire calculée par la formule :
$$\text{Comment lines} / (\text{Lines of code} + \text{Comment lines}) * 100$$
Densité de 50% = autant de lignes de code que de commentaires
Densité de 100% = le fichier ne contient que des commentaires
 - **Commented-out LOC** = nombre de lignes de code commentées.

3. Mesures pour la duplication du code :

- Les variables utilisés pour le calcul :
 - **Duplicated blocks** = nombre de blocs de lignes dupliques. Un bloc de code est considéré comme dupliqué s'il y a au moins 100 tokens successifs dupliques sur 10 lignes de code (PHP, JavaScript)
 - **Duplicated files** = nombre de fichiers impliqués dans des duplications
 - **Duplicated lines** = nombre de lignes impliquées dans des duplications
 - **Duplicated lines (%)** = densité de duplication calculée par la formule : $\text{Duplicated lines} / \text{Lines} * 100$

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Exemples de métriques avec Sonarqube

4. Mesures pour les tests :

- Les variables utilisés pour faire cette mesure :
 - **Nombre de tests unitaires** (unit tests)
 - **nombre de tests unitaires** qui ont échoué (unit test errors)
 - **nombre d'exception** lors de l'exécution (unit test failures)
 - **nombre de lignes non couvertes** (uncovered lines)
 - **nombre de conditions non couvertes** (uncovered conditions)
 - **couverture** : calcul = $(CT + CF + LC) / (2*B + EL)$

CT : condition évaluées à true au moins une fois

CF : conditions évaluées à false au moins une fois

LC : lignes couvertes

B : nombre total de conditions

EL : nombre total de lignes exécutables

5. Mesures de complexité :

- Les variables utilisés pour faire cette mesure :
 - **Complexity** = Complexité totale du projet
 - **Complexity / method** = Complexité moyenne par fonction
 - **Complexity / file** = Complexité moyenne par fichier
 - **Complexity / class** = Complexité moyenne par classe
- Le calcul de la complexité d'une méthode se base sur la complexité cyclomatique. Elle est minimum de 1 (1 pour la méthode). Elle est incrémentée de 1 pour chaque if, case, for, while, throw, catch, return (sauf s'il est en dernière instruction de la fonction), &&, ||, opérateur conditionnel ternaire

CHAPITRE 2

Manipuler l'outil de mesure de la qualité du code (SonarQube)

1. Notions des métriques de la qualité du code
2. Présentation de SonarQube
- 3. Installation de SonarQube**
4. Intégration de SonarQube avec les outils ALM et Configuration
5. Analyse de code source avec SonarQube



02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Installation de sonarqube

Installation de SonarQube :

1. Prérequis:

- **Prérequis côté serveur** : installer JDK sur <https://www.oracle.com/java/technologies/downloads/>
- **Prérequis de base de données**: Postgres ou Oracle ou SqlServer
- **Prérequis côté client** : navigateur web récent
- **JavaScript activé.**

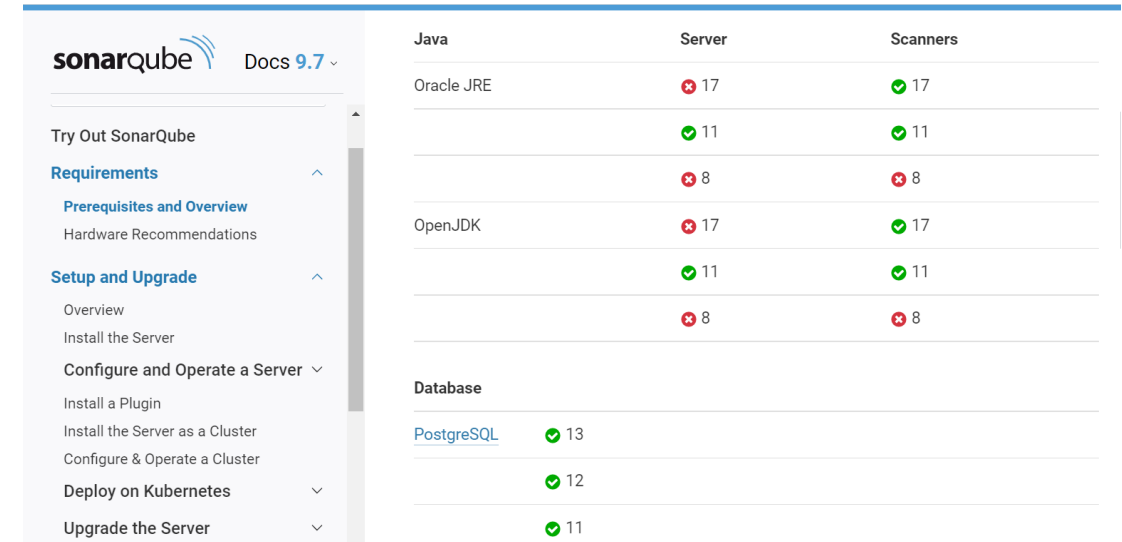
On peut vérifier les prérequis ainsi que leurs versions valables en visitant ce lien :

<https://docs.sonarqube.org/latest/requirements/requirements/>

2. Télécharger **Sonarqube community** sur:

<https://www.sonarqube.org/downloads/>

3. Décompressez le fichier dans n'importe quel dossier (le nom du dossier de ne doit commencer par un chiffre)



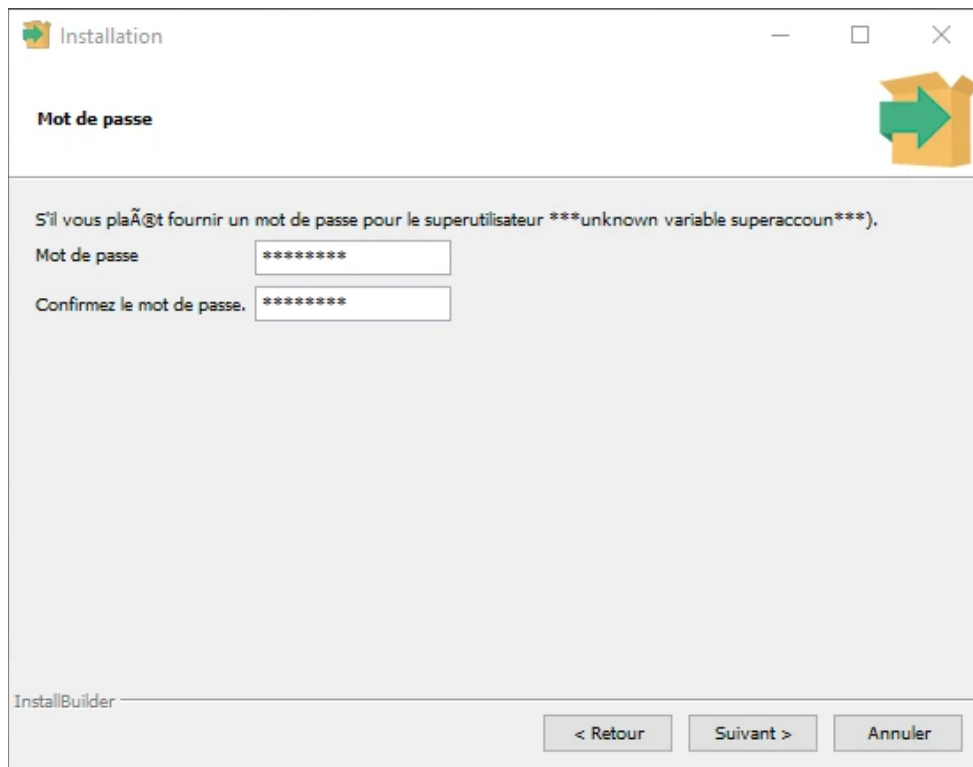
Java	Server	Scanners
Oracle JRE	✗ 17	✓ 17
	✓ 11	✓ 11
	✗ 8	✗ 8
OpenJDK	✗ 17	✓ 17
	✓ 11	✓ 11
	✗ 8	✗ 8
Database		
PostgreSQL	✓ 13	
	✓ 12	
	✓ 11	

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Installation de sonarqube

Installation et configuration du serveur de bases de données Postgres :

En suivant les instructions de l'installation, configurez le mot de passe (ex.:12345) ainsi que le port du SGBD (ex.: 5432) :



Mot de passe

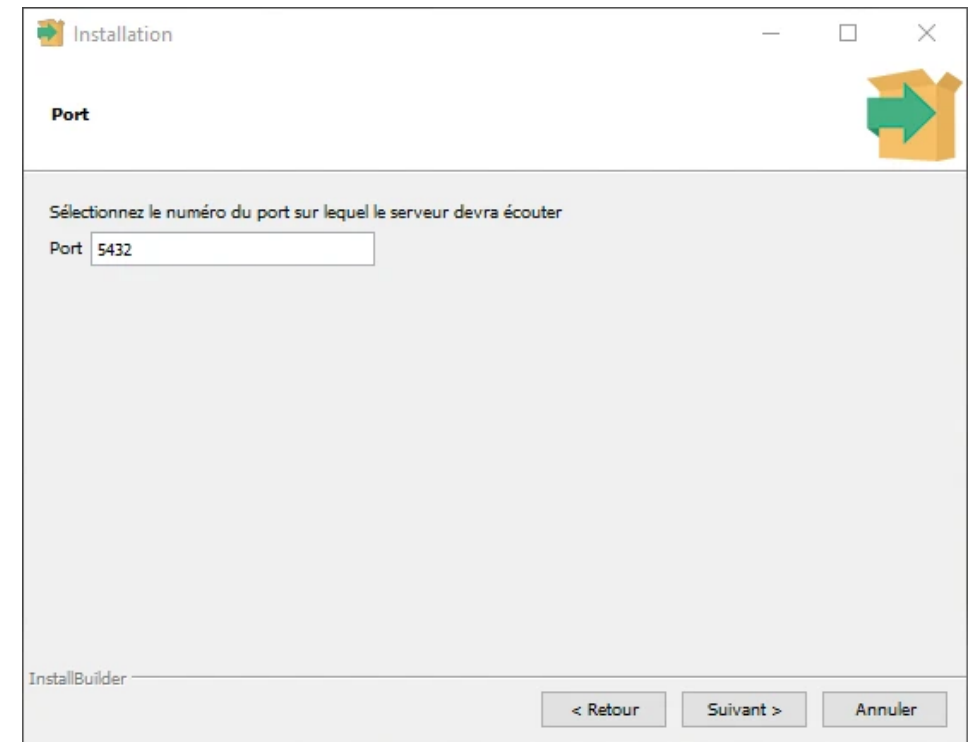
S'il vous plaît fournir un mot de passe pour le superutilisateur ***unknown variable superaccoun***).

Mot de passe

Confirmez le mot de passe.

InstallBuilder

< Retour Suivant > Annuler



Port

Sélectionnez le numéro du port sur lequel le serveur devra écouter

Port

InstallBuilder

< Retour Suivant > Annuler

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Installation de sonarqube

Configuration de SonarQube :

1. Après avoir décompressé le fichier zip téléchargé: sonarqube-9.7.1.62043.zip (dans notre cas)
2. Accédez au dossier config et ouvrez le fichier « sonar.properties »:
 - a. Décommentez et modifiez les lignes suivantes, ainsi:

```
sonar.jdbc.username=postgres
sonar.jdbc.password=12345
.....
sonar.jdbc.url=jdbc:postgresql://localhost:5432/postgres
... .
sonar.web.host=0.0.0.0
.....
sonar.web.port=9000
... .
```


02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Installation de sonarqube

Démarrage de SonarQube :

1.Dans le dossier bin choisir le dossier relative à votre système d'exploitation

) sonarqube-9.5.0.56709 > bin

Nom	Modifié le	Type	Taille
jsw-license	10/06/2022 06:14	Dossier de fichiers	
linux-x86-64	10/06/2022 06:14	Dossier de fichiers	
macosx-universal-64	10/06/2022 06:14	Dossier de fichiers	
windows-x86-64	10/06/2022 06:14	Dossier de fichiers	

2.Cliquer sur **startSonar.bat** pour le démarrer :

Nom	Modifié le	Type
lib	06/11/2022 12:34	Dossier de fichiers
SonarService.bat	28/10/2022 06:26	Fichier de commande
StartSonar.bat	28/10/2022 06:26	Fichier de commande

3. Aller dans un navigateur et taper l'url **localhost:9000** pour afficher l'interface admin de **SonarQube**:

4.Connectez avec les identifiants suivants:

- login: **admin**
- password: **admin**

CHAPITRE 2

Manipuler l'outil de mesure de la qualité du code (SonarQube)

1. Notions des métriques de la qualité du code
2. Présentation de SonarQube
3. Installation de SonarQube
- 4. Analyse de code source avec SonarQube**
5. Intégration de SonarQube avec les outils ALM et Configuration

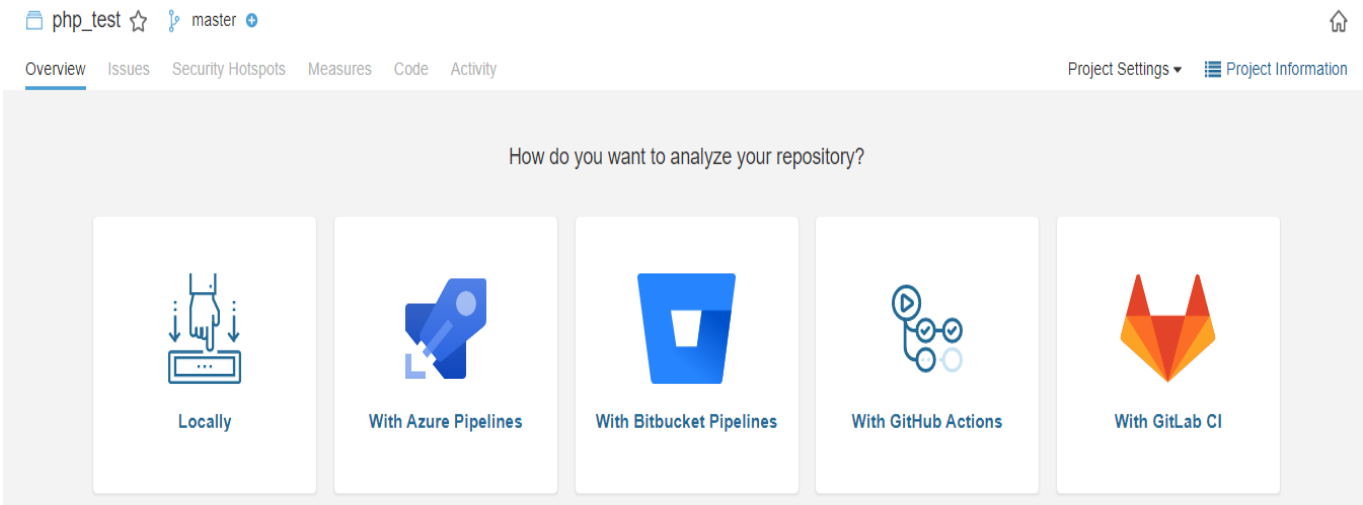


02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

- Maintenant que vous êtes connecté à votre instance **SonarQube** locale, analysons un projet :
 - Cliquez sur le bouton **Créer un nouveau projet** (vous pouvez choisir entre **locally** ou **importer des logiciels des versions**)
 - Donnez à votre projet une **clé de projet** et un **nom complet**



Create a project

All fields marked with * are required

Project display name *

Up to 255 characters. Some scanners might override the value you provide.

Project key *

The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Set Up

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

3.Sous **Fournir un jeton(token)**, sélectionnez **Générer un jeton**. Donnez un nom à votre jeton, cliquez sur le bouton **Générer**, puis sur **Continuer**.

php_test ☆ master +

Overview

Issues

Security Hotspots

Measures

Code

Activity

Analyze your project

We initialized your project on SonarQube, now it's up to you to launch analyses!

1 Provide a token

Generate a token

Generate

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point of time in your [user account](#).

php_test ☆ master +

Overview

Issues

Security Hotspots

Measures

Code

Activity

Analyze your project

We initialized your project on SonarQube, now it's up to you to launch analyses!

1 Provide a token

php_test: 375adb16342e02546551

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point of time in your [user account](#).

Continue

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

4.Par la suite ,choisir le langage de votre projet et le système d'exploitation :

php_test ☆ master +

Overview

Issues

Security Hotspots

Measures

Code

Activity

Analyze your project

We initialized your project on SonarQube, now it's up to you to launch analyses!

1 Provide a token

2 Run analysis on your project

What option best describes your build?

Maven

Gradle

.NET

Other (for JS, TS, Go, Python, PHP, ...)

php_test ☆ master +

Overview

Issues

Security Hotspots

Measures

Code

Activity

1 Provide a token

2 Run analysis on your project

What option best describes your build?

Maven

Gradle

.NET

Other (for JS, TS, Go, Python, PHP, ...)

What is your OS?

Linux

Windows

macOS

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

5.Ensuite ,télécharger et installer **SonarScanner** , selon votre **SE** sur : <https://docs.sonarqube.org/latest/analysis/scan/sonarscanner>

SonarScanner

By [SonarSource](#) | GNU LGPL 3 | [Issue Tracker](#)

4.7

[Show more versions](#)

2022-02-22

Ease import of custom certificates with the Docker image,
update embedded JRE 11

[Linux 64-bit](#) [Windows 64-bit](#) [Mac OS X 64-bit](#) [Docker](#)

[Any \(Requires a pre-installed JVM\)](#) [Release notes](#)

The SonarScanner is the scanner to use when there is no specific scanner for your build system.

Le SonarScanner est un outil utilisé par **SonarQube** pour scanner votre projet vers **SonarQube** pour que ce dernier puisse lire le langage de programmation utilisé.

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube : Configurer sonnar-scanner

1. Décompressez le fichier téléchargé dans un répertoire de votre choix ;
2. Sous le dossier « **config** », accédez au fichier « **sonar-scanner.properties** » et décommentez la ligne suivante :

```
#----- Default SonarQube server  
sonar.host.url=http://localhost:9000
```

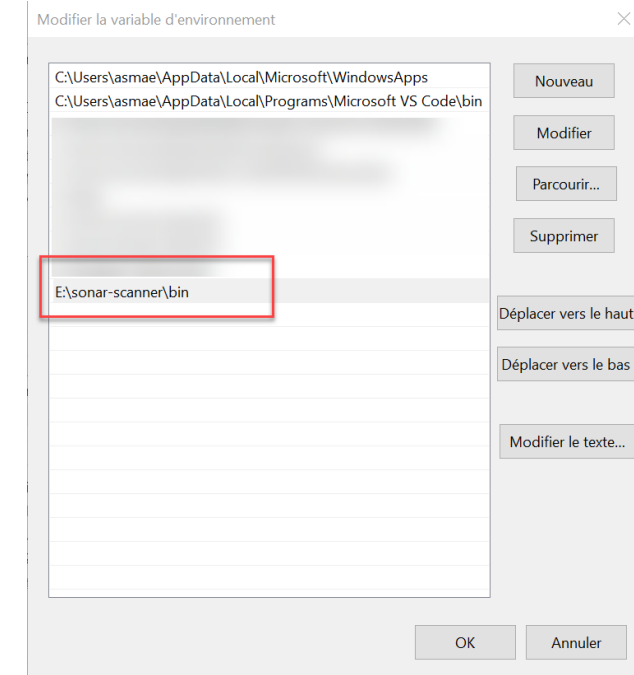
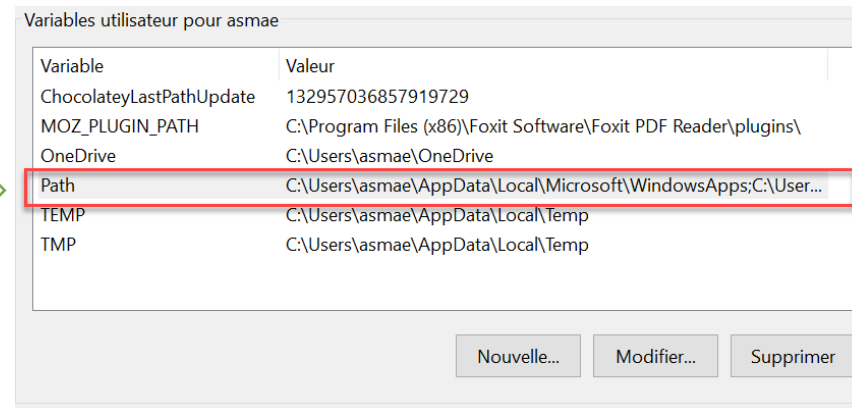
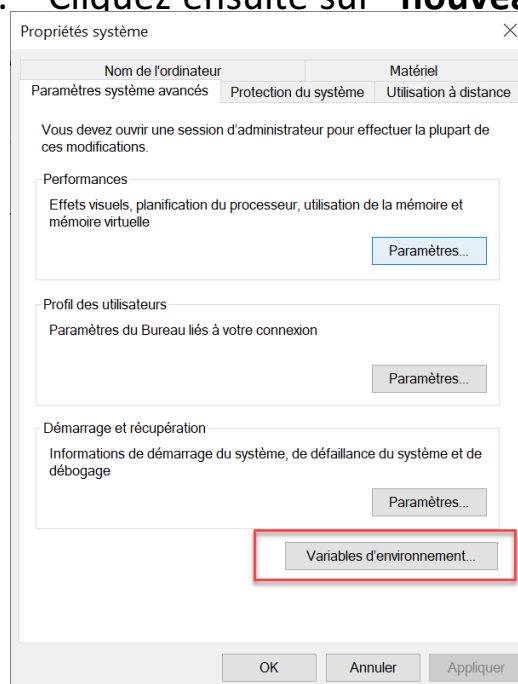
02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube : Configurer sonnar-scanner

Afin de faciliter l'appel au sonar scanner, il vaut mieux l'ajouter au path windows, pour cela :

1. Bouton doit sur l'icone windows, puis accédez à **"système"**
2. Sur le menu à droite, accédez à **"Paramètres avancés du système"**
3. Cliquez sur **"Variables d'environnement"**
4. Sélectionnez **"path"** puis cliquez sur **"modifier"**
5. Cliquez ensuite sur **"nouveau"** et tapez le chemin vers le dossier bin de sonnar-scanner;



02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube : Configurer sonnar-scanner

Pour commencer l'analyse du projet:

1. Ajouter un nouveau fichier à votre projet; le fichier devra prendre le nom : **sonar-project.properties** et doit avoir le contenu suivant :

```
# must be unique in a given SonarQube instance  
sonar.projectKey=php_test
```

La propriété **Sonar.projectkey** devra avoir le nom de votre projet

```
# --- optional properties ---
```

```
# defaults to project key  
#sonar.projectName=My project  
# defaults to 'not provided'  
#sonar.projectVersion=1.0
```

```
# Path is relative to the sonar-project.properties file. Defaults to .  
#sonar.sources=.
```

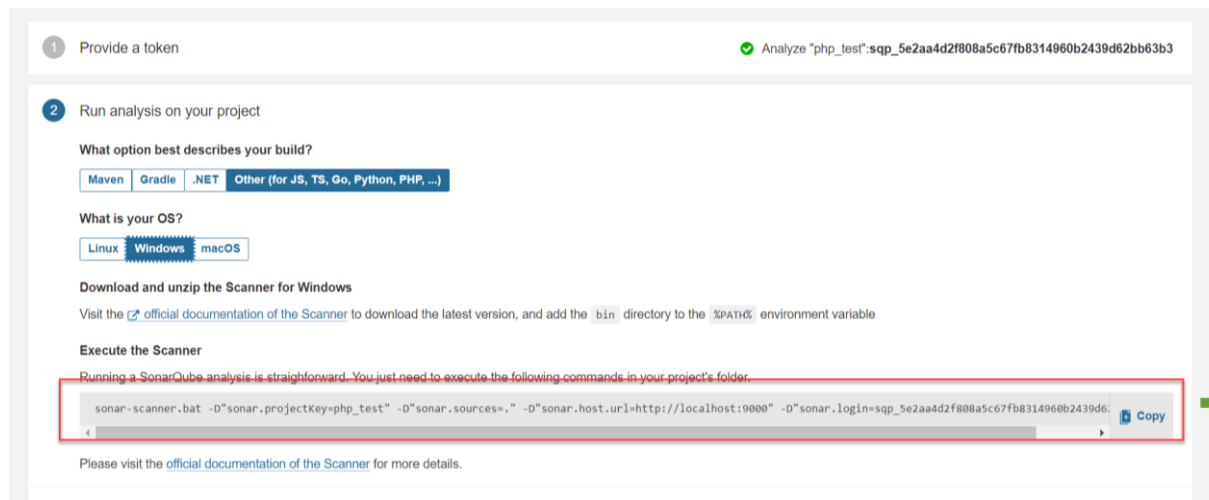
```
# Encoding of the source code. Default is default system encoding  
#sonar.sourceEncoding=UTF-8
```

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

Copiez la commande proposée par Sonarqube et collez la dans le terminal de visual studio :



1 Provide a token ✓ Analyze "php_test":sqp_5e2aa4d2f808a5c67fb8314960b2439d62bb63b3

2 Run analysis on your project

What option best describes your build?

Maven Gradle .NET Other (for JS, TS, Go, Python, PHP, ...)

What is your OS?

Linux Windows macOS

Download and unzip the Scanner for Windows

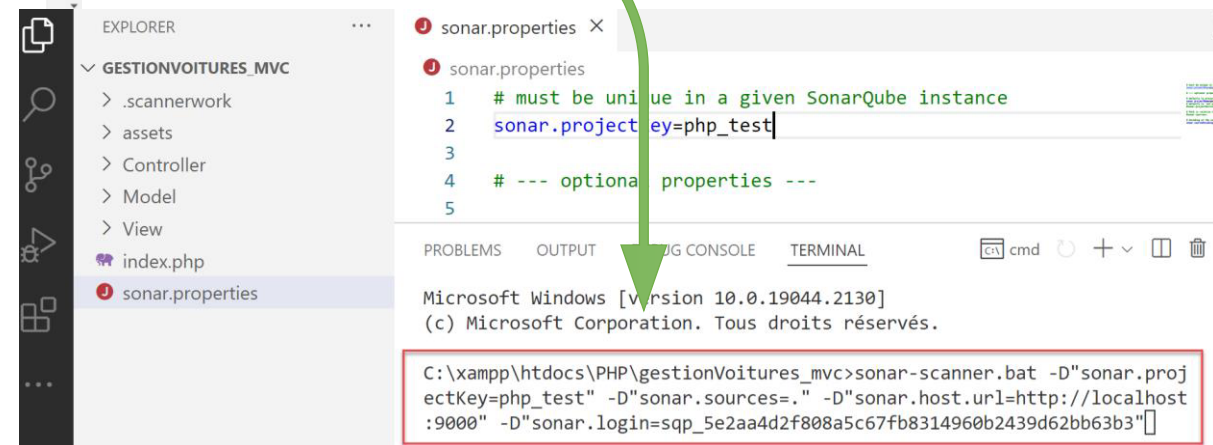
Visit the [official documentation of the Scanner](#) to download the latest version, and add the `bin` directory to the `PATH` environment variable

Execute the Scanner

Running a SonarQube analysis is straightforward. You just need to execute the following commands in your project's folder.

```
sonar-scanner.bat -D"sonar.projectKey=php_test" -D"sonar.sources=." -D"sonar.host.url=http://localhost:9000" -D"sonar.login=sqp_5e2aa4d2f808a5c67fb8314960b2439d62bb63b3"
```

Please visit the [official documentation of the Scanner](#) for more details.



EXPLORER

- ✓ GESTIONVOITURES_MVC
 - > .scannerwork
 - > assets
 - > Controller
 - > Model
 - > View
 - index.php
 - sonar.properties

sonar.properties

```
1 # must be unique in a given SonarQube instance
2 sonar.projectKey=php_test
3
4 # --- optional properties ---
5
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Microsoft Windows [Version 10.0.19044.2130]
(c) Microsoft Corporation. Tous droits réservés.

```
C:\xampp\htdocs\PHP\gestionVoitures_mvc>sonar-scanner.bat -D"sonar.projectKey=php_test" -D"sonar.sources=." -D"sonar.host.url=http://localhost:9000" -D"sonar.login=sqp_5e2aa4d2f808a5c67fb8314960b2439d62bb63b3"
```

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

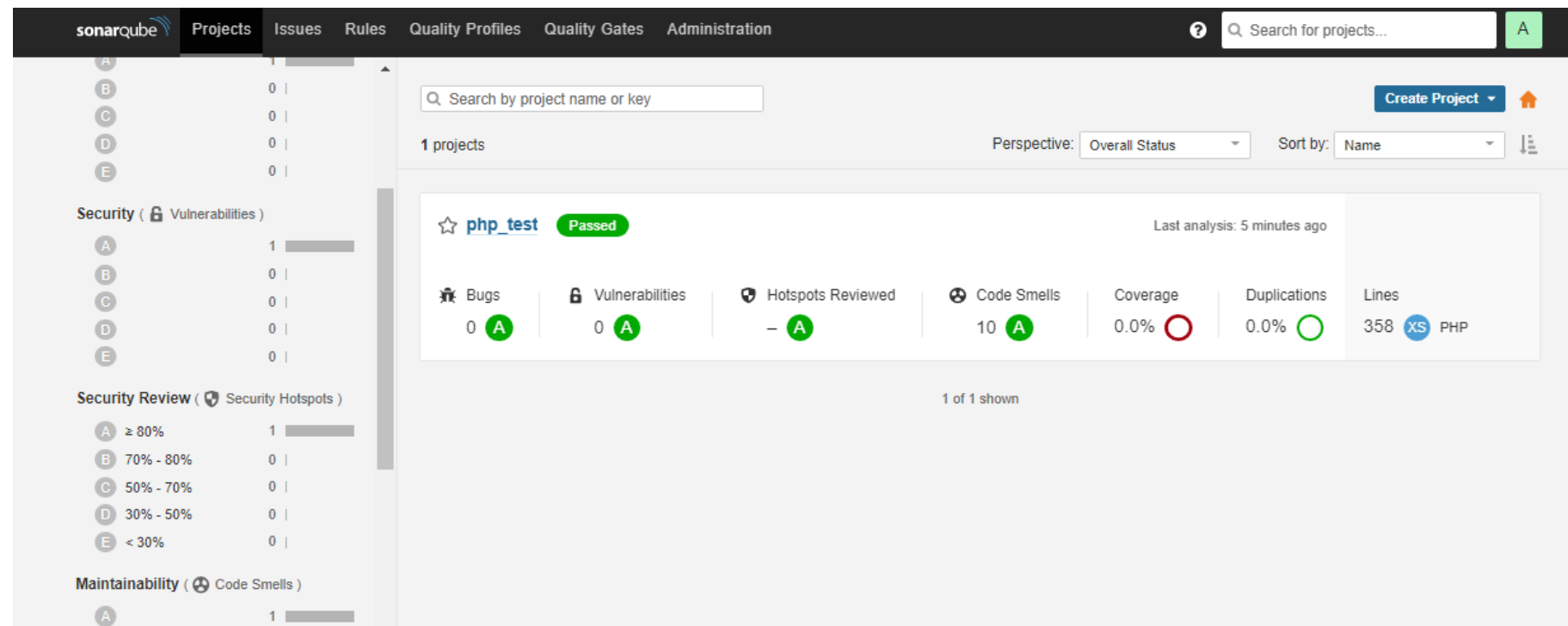
Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

8. A la fin de l'analyse un url s'affiche pour voir les résultats ,copier le et accéder via un navigateur :

More about the report processing at <http://localhost:9000/api/ce/task?id=AYImYtoQQCeb5V2Rt8ou>
Analysis total time: 1:35.634 s

Le tableau du bord de l'analyse du code s'affiche:

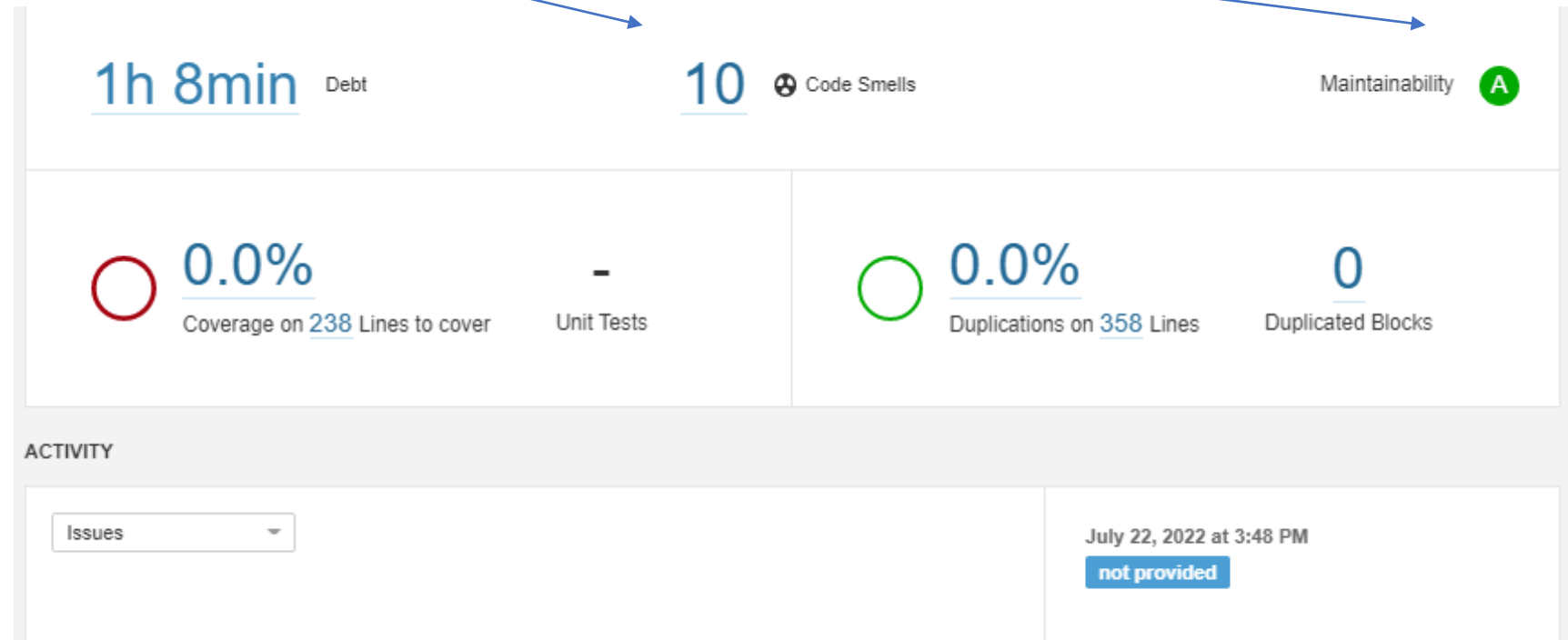


02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

- On peut accéder à chaque métrique en cliquant sur son nom :
- Les numéros sont le nombre d'erreurs dans le code



02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Analyser le code avec SonarQube

Analyser un projet avec SonarQube:

- On peut voir les erreurs dans le code et avec la suggestion de correction:

```
body {
  font-size: 17px;
  font-family: arial;
```

Unexpected missing generic font family Why is this an issue?

Bug
 Major
 Open
 Not assigned
 1min effort
 Comment

```
function inverse($x) {
  if (!$x) {
    throw new Exception('Division by zero.');
```

Define and throw a dedicated exception instead of using a generic one. Why is this an issue? 16 minutes ago ▾ L11 🔗

Code Smell
 Major
 Open
 Not assigned
 20min effort
 Comment
 cwe, error-handling ▾

```
$colors = [
  1 => 'red',
  2 => 'green',
  3 => 'blue',
];

// echo $colors[1];
```

Remove this commented out code. Why is this an issue? 16 minutes ago ▾ L36 🔗

Code Smell
 Major
 Open
 Not assigned
 5min effort
 Comment
 unused ▾

```
php2 /index.html
<!DOCTYPE html>
<html>
```

Add "lang" and/or "xml:lang" attributes to this "<html>" element Why is this an issue? 6 minutes ago ▾ L2 🔗

Bug
 Major
 Open
 Not assigned
 Comment
 accessibility, wcag2-a ▾

CHAPITRE 2

Manipuler l'outil de mesure de la qualité du code (SonarQube)

1. Notions des métriques de la qualité du code
2. Présentation de SonarQube
3. Installation de SonarQube
4. Analyse de code source avec SonarQube
5. **Intégration de SonarQube avec les outils ALM et Configuration**



02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

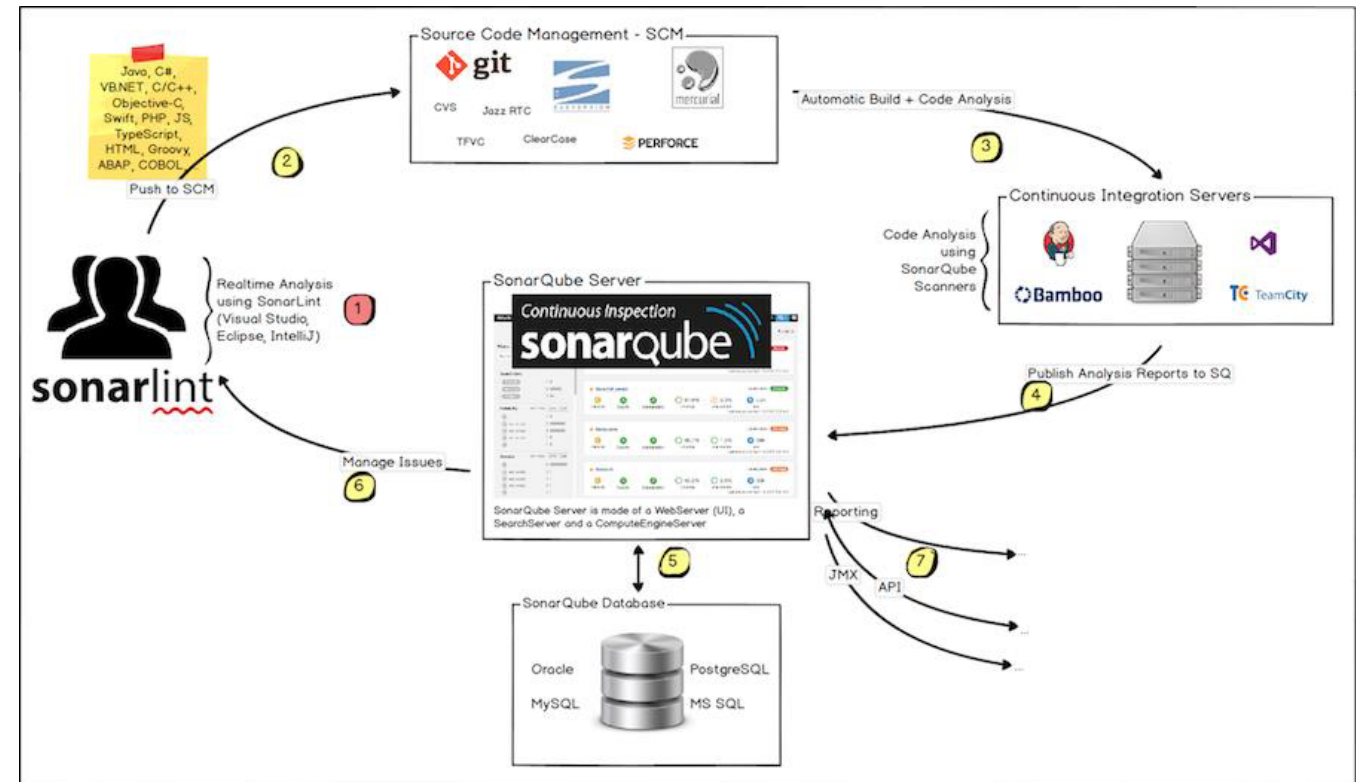
Intégration de SonarQube avec les outils ALM

Intégration de SonarQube avec les outils ALM :

Le schéma suivant montre comment **SonarQube** s'intègre à d'autres outils **ALM**¹ dans un grand projet d'équipe e où les différents composants de **SonarQube** sont utilisés

¹ L'application life cycle

management ou **ALM** :est le processus global de gestion du cycle de vie d'un logiciel. C'est l'ensemble des moyens nécessaires au développement ou à la maintenance d'une application. Cela concerne les activités d'ingénierie logicielle comme les activités de gestion de projet.



Source : <https://docs.sonarqube.org/>

02-Manipuler l'outil de mesure de la qualité du code (SonarQube) :

Intégration de SonarQube avec les outils ALM

Intégration avec Application Life cycle management:

- Les rôles dans le schéma selon les numéros :

1. Les développeurs codent dans leurs IDE et utilisent **SonarLint** pour exécuter une analyse locale.
2. Les développeurs poussent leur code dans leur **SCM** préféré : **github**, **gitlab**...
3. Le **serveur d'intégration continue** déclenche une construction automatique et l'exécution du **SonarScanner** requis pour exécuter l'analyse **SonarQube**.
4. Le rapport d'analyse est envoyé au serveur **SonarQube** pour traitement.
5. **SonarQube** Server traite et stocke les résultats du rapport d'analyse dans la base de données **SonarQube** et affiche les résultats dans l'interface utilisateur.
6. Les développeurs examinent, commentent, contestent leurs problèmes pour gérer et réduire leur dette technique via l'interface utilisateur **SonarQube**.
7. Les managers reçoivent les rapports de l'analyse. Les opérateurs utilisent des API pour automatiser la configuration et extraire les données de **SonarQube**. Les opérateurs utilisent JMX pour surveiller le serveur **SonarQube**.

SonarLint est une extension IDE gratuite et open source qui identifie et vous aide à résoudre les problèmes de qualité et de sécurité pendant que vous codez. Comme un correcteur orthographique, SonarLint corrige les défauts et fournit des commentaires en temps réel et des conseils de correction clairs pour fournir un code propre dès le départ. Il supporte presque tous les IDE comme vscode ,eclipse ,JetBrains..

- Quelques règles avant la configuration de SonarQube :

- La plate-forme **SonarQube** ne peut pas avoir plus d'un serveur **SonarQube** et plus d'une base de données .
- Pour des performances optimales, chaque composant (serveur, base de données, analyseurs) doit être installé sur une machine distincte et la machine serveur doit être dédiée.
- Les **SonarScanners** évoluent en ajoutant des machines.
- Toutes les machines doivent être synchronisées.
- Le serveur **SonarQube** et la base de données **SonarQube** doivent être situés sur le même réseau.
- Les **SonarScanners** n'ont pas besoin d'être sur le même réseau que le serveur **SonarQube**.
- Il n'y a pas de communication entre les **SonarScanners** et la base de données **SonarQube**.